

Assignment 1 – PS4 Defence Drone

Greedy Best-First Search (GBFS) and A* Search Analysis

1. Problem Statement

An autonomous UAV must navigate from Start node (0,0) to Goal node (6,7) while avoiding No-Fly Zones (N) and accounting for Weather Hazard regions (W). The objective is to evaluate Greedy Best-First Search (GBFS) using two heuristics and compare its performance with A* Search.

2. Environment Representation

Grid Size: 8×8. Symbols used: S=Start, E=Goal, .=Open Cell, W=Weather Hazard, N=No-Fly Zone. Movement is restricted to North, East, South and West.

3. PEAS Analysis

Performance: Reach the target with minimum risk and cost.

Environment: 8×8 grid world.

Actuators: Move North, East, South, West.

Sensors: Current position and neighboring cells.

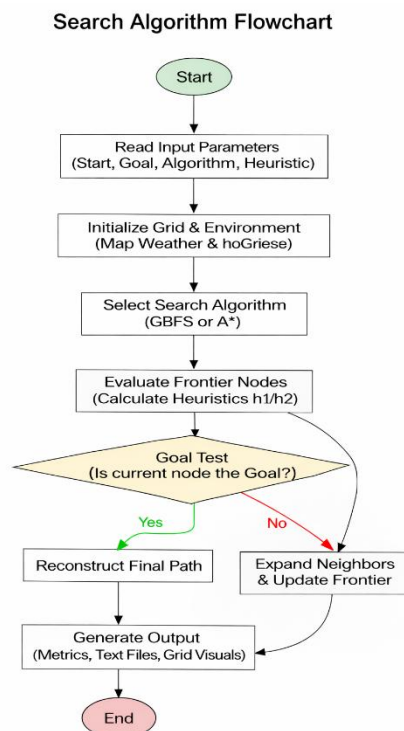
4. Heuristic Functions

h1: Euclidean Distance.

h2: Bounding-Box Risk Weighted Heuristic which combines Manhattan distance with average future path risk.

5. Execution Flow

Read Input → Validate Input → Build Grid → Select Search Algorithm → Expand Nodes → Compute Heuristic → Goal Test → Reconstruct Path → Generate Output.



6. Initial Environment and Planned Route

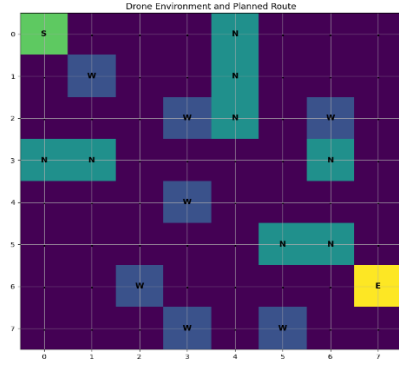
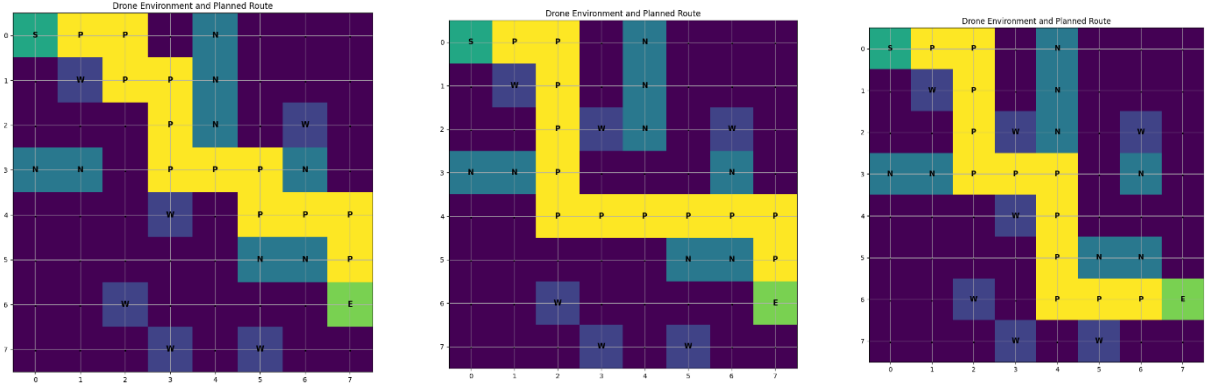


Figure 6 (a): Initial Environment and Planned Route shows the defense drone environment. The start node S is located at (0,0) and the goal node E at (6,7). No-Fly Zones (N) are treated as obstacles while Weather Hazard cells (W) have higher traversal cost.

7. Final Path Visualization



| GBFS-h1 Final Path (Cost = 33) Figure 7(a) | GBFS-h2 Final Path (Cost = 19) Figure 7(b) | A*-h1 Final Path (Cost = 16) Figure 7(c) |

The final paths generated by GBFS-h1, GBFS-h2, and A*-h1 illustrate how each search strategy navigates the environment from the Start node to the Goal node while handling obstacles and hazard traversal costs. GBFS-h1 produced the fastest solution in terms of node expansions (14 nodes) and runtime (0.130 ms); however, due to the greedy nature of the Euclidean distance heuristic ($\$h_1\$$), it pulled the drone tightly along the geometric diagonal. This forced the path into sub-optimal detours and through a Weather Hazard cell, resulting in a high path cost of 33, though it successfully avoided entering any No-Fly Zone cells. GBFS-h2 generated a safer, more balanced route by utilizing the Bounding-Box Risk-Weighted Heuristic, bypassing severe risks entirely to achieve a reduced path cost of 19. A*-h1 successfully discovered the mathematically optimal path with the lowest possible path cost of 16 by perfectly balancing actual past cost and future heuristic estimates, entirely avoiding both No-Fly Zones and Weather Hazard cells. These visualizations clearly highlight the fundamental trade-off between greedy computational efficiency and true path optimality among the evaluated search algorithms.

8. Comparative Analysis

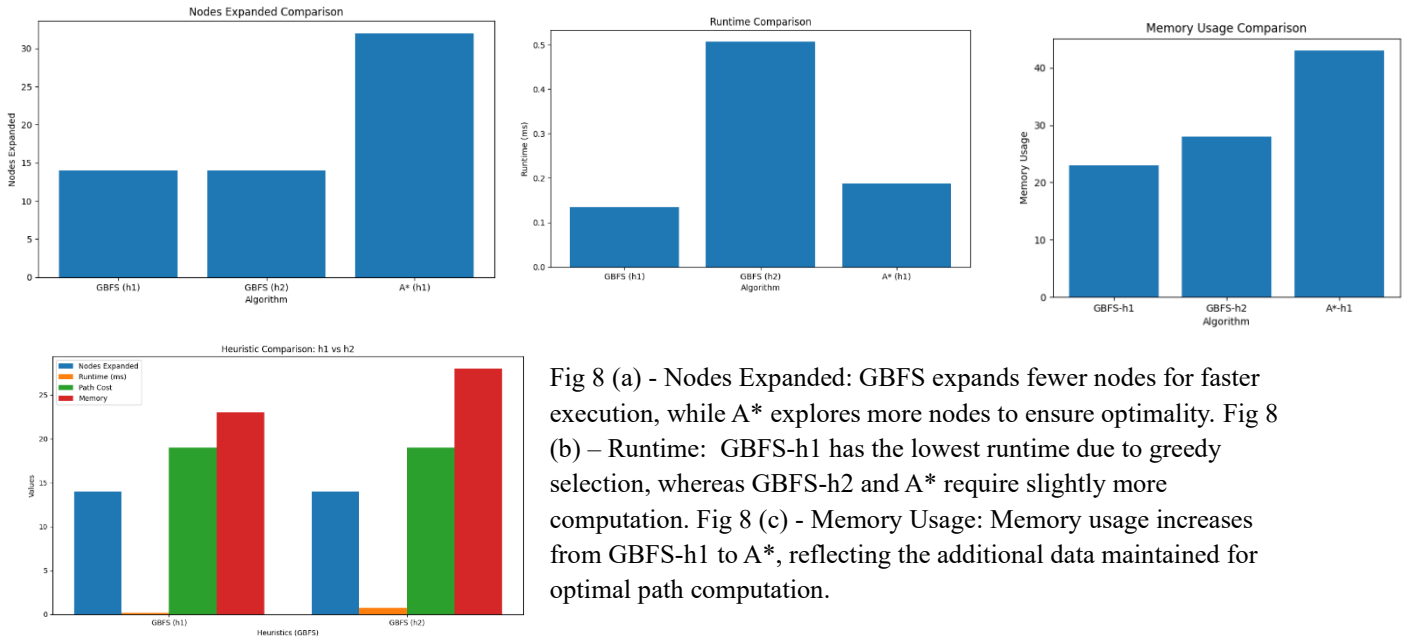


Fig 8 (a) - Nodes Expanded: GBFS expands fewer nodes for faster execution, while A* explores more nodes to ensure optimality. Fig 8 (b) – Runtime: GBFS-h1 has the lowest runtime due to greedy selection, whereas GBFS-h2 and A* require slightly more computation. Fig 8 (c) - Memory Usage: Memory usage increases from GBFS-h1 to A*, reflecting the additional data maintained for optimal path computation.

9. Experimental Results

Metric	Description	GBFS-h1	GBFS-h2	A*-h1
Nodes Expanded	Total explored nodes	14	14	32
Runtime (ms)	Execution time	0.130388	0.636698	0.317236
Memory Usage	OPEN + CLOSED size	26	32	51
Total Path Cost	Sum of transition costs	33	19	16
Path Length	Total moves	13	13	13
Initial Heuristic Value	Start node heuristic value	9.22	29.95	9.22

The experimental results highlight the trade-off between efficiency and solution quality. Both GBFS variants expanded 14 nodes and reached the goal in 13 moves. GBFS-h1 achieved the fastest runtime (0.130388 ms) and lowest memory usage (26), but produced the highest path cost (33) and crossed two No-Fly Zones. GBFS-h2 required more runtime and memory but improved path quality by reducing the cost to 19 and avoiding all No-Fly Zones. A*-h1 expanded the most nodes (32) and used the most memory (51), yet produced the optimal path with the lowest cost (16) while avoiding all hazards.

10. Complexity Analysis

Greedy Best-First Search (GBFS) uses a priority queue ordered by the heuristic value $h(n)$. Its time complexity is approximately $O((V+E)\log V)$ and space complexity is $O(V)$. In this experiment, GBFS expanded only 14 nodes, making it computationally efficient.

A* Search uses the evaluation function $f(n)=g(n)+h(n)$, combining path cost and heuristic information. Its time complexity is also $O((V+E)\log V)$ with $O(V)$ space complexity. Although A* expanded 32 nodes and used more memory, it generated the optimal path with the lowest cost.

Algorithm	Time Complexity	Space Complexity
GBFS-h1	$O((V+E) \log V)$	$O(V)$

GBFS-h2	$O((V+E) \log V)$	$O(V)$
A*-h1	$O((V+E) \log V)$	$O(V)$

11. Trap Analysis

No dead-end states, local minima, or infinite loops were encountered. All algorithms successfully reached the goal without requiring trap-recovery mechanisms.

12. Alternate Problem Modelling

The environment can alternatively be modelled as a graph, where each grid cell is represented as a node and adjacent traversable cells form edges. This representation improves scalability and supports advanced search algorithms such as Dijkstra's and A*, but requires additional memory to store graph structures. For the current 8×8 environment, the grid representation is simpler and more efficient.

13. Completeness and Optimality Comparison

Algorithm	Complete	Optimal
GBFS	Yes (for this finite grid)	No
A*	Yes	Yes

GBFS successfully reached the target in all experiments and is complete for this finite environment. However, it does not guarantee an optimal solution because the heuristic alone determines node selection. This is reflected in the higher path costs produced by GBFS-h1 (33) and GBFS-h2 (19). A* is both complete and optimal because it considers the accumulated path cost and heuristic estimate simultaneously, enabling it to find the minimum-cost path of 16.

14. Conclusion

All three search configurations successfully reached the target node. GBFS-h1 proved to be highly computationally efficient, achieving the fewest node expansions (14) and the lowest memory footprint (26). However, because the Euclidean distance ignores environmental obstacles, it guided the drone into a high-cost trajectory close to obstacle boundaries and through a Weather Hazard zone, accumulating a path cost of 33. GBFS-h2 expanded the same number of nodes (14) but significantly improved route quality, reducing the path cost to 19, by adjusting its frontier evaluation based on forward-looking risk boundaries. A* expanded the most nodes (32) and required a higher memory footprint (51) but guaranteed safety and economic efficiency, producing the optimal solution with the lowest path cost of 16 while steering completely clear of all hazards. These results demonstrate that while GBFS variants are ideal for rapid, resource-constrained execution, A* remains necessary when path quality and absolute risk avoidance are the primary objectives.